

Module: Structures in C

What is a Structure?

- A **structure** (**struct**) is a user-defined data type
- It groups **variables of different types** under one name

```
struct Person {  
    char name[50];  
    int age;  
    float height;  
};
```

Declaring Structure Variables

```
struct Person p1;
```

- Access fields using the **dot operator (.)**

```
p1.age = 25;  
printf("%d", p1.age); // 25
```

Initialization of Structures

PROF

Basic initialization

```
struct Person p1 = {"Alice", 25, 1.70};
```

Designated initialization

```
struct Person p2 = {  
    .age = 30,  
    .height = 1.80,  
    .name = "Bob"  
};
```

Using `typedef`

```
// Define a new type using typedef
typedef struct {
    char name[50];
    int age;
    float height;
} Person ;

int main() {
    Person p1 = {"Bob", 30, 1.80};

    Person p2 = {.age = 25,
                .height = 1.70,
                .name = "Alice"};

    ...
}
```

Operations on Structures

- Field access: `p.age`
- Assignment: `p2 = p1` (copies all fields)
- Cannot compare with `==`

Structure with Same-Type Fields

Structures can also group homogeneous data

```
typedef struct {
    int numerator, denominator;
} Fraction;
```

Example usage:

```
Fraction f = {1, 2}; // represents 1/2
```

□ Useful for modeling mathematical objects

Structures and Functions

- Can pass:

- by value (copy)
- by pointer (modify original)

```
void birthday(Person *p) {  
    p->age++;  
}
```

Pointers to Structures (-> operator)

- When using a pointer to a structure:

```
Person *p;
```

Two equivalent ways to access a field:

```
(*p).age = 30; // dereference then access
```

```
p->age = 30; // shorthand notation
```

□ `p->age` means:

- go to the structure pointed by `p`
- access field `age`

□ It is equivalent to:

```
(*p).age
```

PROF

Arrays of Structures

```
Person people[3];
```

Example: Deck of French Cards

- value + suit representation
-

Card Structure (Strings)

```
typedef struct {  
    char value[6];  
    char suit[10];  
} Card;
```

Deck Definition

```
#define DECK_SIZE 52  
Card deck[DECK_SIZE];
```

Creating the Deck

```
#include <string.h>  
  
void createDeck(Card deck[]) {  
    char *values[] = {  
        "Ace", "2", "3", "4", "5", "6", "7",  
        "8", "9", "10", "Jack", "Queen", "King"  
    };  
  
    char *suits[] = {  
        "Hearts", "Diamonds", "Clubs", "Spades"  
    };  
  
    int index = 0;  
  
    for (int s = 0; s < 4; s++) {  
        for (int v = 0; v < 13; v++) {  
            strcpy(deck[index].value, values[v]);  
            strcpy(deck[index].suit, suits[s]);  
            index++;  
        }  
    }  
}
```

PROF

Why `strcpy` is Used

- `deck[index].value` is an **array of characters**
- `values[v]` is a **pointer to a string**

❑ Direct assignment is NOT allowed:

```
// deck[index].value = values[v]; // ERROR
```

❑ Correct approach:

```
strcpy(deck[index].value, values[v]);
```

Explanation

- Arrays cannot be assigned directly
- `strcpy` copies characters **one by one**
- Ensures the string is stored inside the structure

Shuffling the Deck

```
#include <stdlib.h>

void shuffleDeck(Card deck[]) {
    for (int i = 0; i < 52; i++) {
        int j = rand() % 52;

        Card temp = deck[i];
        deck[i] = deck[j];
        deck[j] = temp;
    }
}
```

PROF

Enumerations (`enum`)

```
enum Suit {
    HEARTS = 0,
    DIAMONDS = 1,
    CLUBS = 2,
    SPADES = 3
};
```

`typedef` with Enum

```
typedef enum {  
    ACE=1, TWO, THREE, FOUR, FIVE,  
    SIX, SEVEN, EIGHT, NINE, TEN,  
    JACK, QUEEN, KING  
} Value;
```

Summary (Structures)

- Structures group data
- -> simplifies pointer access
- `strcpy` is needed for string copying
- Enums improve readability

Module: Dynamic Memory and Data Structures (Linked Lists)

Heap Memory (Detailed)

- Dynamic allocation area

What Does `malloc` Return?

```
int *p = malloc(5 * sizeof(int)); //memory for an array of five int
```

- Pointer (`void*`) to first allocated byte
- `NULL` if failure

Memory Initialization

- `malloc` → uninitialized
- `calloc` → zeroed

Freeing Memory

```
free(p);
```

Node Structure

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
```

Creating a Node

```
Node* createNode(int value) {
    Node *n = malloc(sizeof(Node));
    n->data = value;
    n->next = NULL;
    return n;
}
```

List Structure

```
typedef struct {
    Node *head;
    Node *tail;
} List;
```

Initialize List

```
void initList(List *l) {
    l->head = NULL;
    l->tail = NULL;
}
```

Insert at End

```
void append(List *l, int value) {
    Node *n = createNode(value);

    if (l->head == NULL) {
        l->head = l->tail = n;
    }
}
```

```
    } else {  
        l->tail->next = n;  
        l->tail = n;  
    }  
}
```

Remove (and Return) the Header

```
Node* removeHead(List *l) {  
    if (l == NULL || l->head == NULL) {  
        return NULL; // empty list  
    }  
  
    Node *removed = l->head;    // store current head  
    l->head = removed->next;    // advance head  
  
    if (l->head == NULL) {  
        l->tail = NULL;        // list is now empty  
    }  
  
    removed->next = NULL;      // detach node  
    return removed;  
}
```

Print List

```
void printList(List *l) {  
    Node *current = l->head;  
  
    while (current != NULL) {  
        printf("%d ", current->data);  
        current = current->next;  
    }  
}
```

PROF

Free List

```
void freeList(List *l) {  
    Node *current = l->head;  
  
    while (current != NULL) {  
        Node *temp = current;  
        current = current->next;  
        free(temp);  
    }  
}
```



```
    }  
  
    l->head = l->tail = NULL;  
}
```

Example Usage

```
List list;  
initList(&list);  
append(&list, 1);  
append(&list, 2);  
printList(&list);  
Node *n = removeHead(&list);  
if (n != NULL) {  
    printf("Removed: %d\n", n->data);  
    free(n);  
}  
printList(&list);
```

Summary

- `malloc` allocates heap memory
- Must free memory manually
- Linked lists are dynamic structures

Final Insight

□ Structures + pointers + heap = dynamic programming power